Autoencoder Neural Networks for Image Generation on MNIST Dataset

Sotiris Manolis, 4444

***Abstract*—This report presents the implementation and evaluation of autoencoder neural networks for image generation using the MNIST dataset. In particular, a convolutional autoencoder architecture is designed and trained to generate realistic handwritten digit images in three scenarios: (1) for each digit/image provided as input, the same digit image is reconstructed, (compared with PCA reconstructions), (2) for each digit/image provided as input, the image of the next digit is generated and (3) for a pair of two digits/images provided as input, the images of the digits of their sum are generated. The performance of the autoencoder is assessed through qualitative visual inspection of the generated images as well as quantitative metrics such as Mean Squared Error (MSE) and classification accuracy using a pre-trained CNN classifier.**

I Introduction

In this report, we explore the use of autoencoder neural networks for image generation using the MNIST dataset of handwritten digits. Autoencoders are a type of neural network which perform semi-supervised learning by encoding input data into a lower-dimensional representation and then decoding it back to reconstruct the original input. The goal of this study is to design and train a convolutional autoencoder that can generate realistic images of handwritten digits for two tasks:

1) Given an input image of a digit, reconstruct the image of the same digit. For this task, we will compare the reconstructed images with PCA reconstructions.
2) Given an input image of a digit, generate the image of the next digit. For example, given an image of the digit '3', the model should generate an image of the digit '4'.
3) Given two input images of digits, generate the images of the digits representing their sum. For example, given images of the digits '9' and '3', the model should generate two images of the digits '1' and '2' (since 9 + 3 = 12).

II Dataset and Preprocessing

As mentioned above, we will use the MNIST dataset, which consists of 60,000 training images and 10,000 test images of handwritten digits (0-9), each of size 28x28 pixels. Also, for feature extraction and dimensionality reduction, we will use Principal Component Analysis (PCA) while retaining 90% of the variance.

Before training the autoencoder, we preprocess the images by normalizing the pixel values to the range [-1, 1], in order to facilitate faster convergence during training and match the output activation function range of the decoder.

Finaly, for each task we create three custom datasets, one for each of the three tasks described in the Introduction section.

- For task 1, the dataset consists of pairs of images where both the input and target images are the same digit image.
- For task 2, the dataset consists of pairs of images where the input image is a digit image and the target image is the image of the next digit.
- For task 3, the dataset consists of pairs of images where the input images are two digit images and the target images are the images of the digits representing their sum. In this case, instead of picking two random numbers, we ensure that each possible sum (0-18) is equally represented in the dataset.

III Autoencoder Architectures

For the implementation of the autoencoders, we used the PyTorch framework. We chose a convolutional autoencoder architecture due to its ability to capture spatial hierarchies in image data. We opted for this architecture instead of a multilayer perceptron (MLP) autoencoder, as convolutional layers are more effective in learning image features, as we understoof from the experiments in the first project.

The basic architecture of the convolutional autoencoder consists of an encoder and a decoder. The **encoder** is a convolutional neural network that compresses the input images into a lower-dimensional latent representation, forcing the model to learn the structural features of the digit images.Its architecture is as follows:

- Input layer: a grayscale image tensor of shape $(N, 1, 28, 28)$, where N is the batch size
- Convolutional layer 1: 16 filters, kernel size 3x3, stride 2, padding 1, followed by ReLU activation, increasing the number of channels to 16 while reducing the spatial dimensions to 14x14
- Convolutional layer 2: 32 filters, kernel size 3x3, stride 2, padding 1, followed by ReLU activation, increasing the number of channels to 32 while reducing the spatial dimensions to 7x7
- Bottleneck: A fully connected linear layer that maps the output of the last convolutional layer to a latent space (e.g. 64 dimensions)

The **decoder** performs the inverse operation, reconstructing the images from the latent dimension back to the original image size. Its architecture is as follows:

- Input layer: the latent vector of shape $(N, latent_dim)$
- Latent Expansion: A fully connected layer which maps the latent vector back to the flattened feature size

(32x7x7). This is then reshaped to a 3D tensor of shape $(N, 32, 7, 7)$.

- Upsampling Blocks: Two transposed convolutional layers (ConvTranspose2d) with ReLU activations to progressively increase the spatial dimensions back to 28x28. The first transposed convolutional layer upsamples from 7x7 to 14x14, and the second upsamples from 14x14 to 28x28.
- Output layer: A final convolutional layer with a Tanh activation function to ensure the output pixel values are in the range [-1, 1].

For tasks 1 and 2 we used the above architecture, combining the encoder and decoder sequentially. The input to the autoencoder is a single image passed through the encoder to produce the latent representation, which is then passed through the decoder to generate the output image. For reconstruction (task 1), the target image is the same as the input image, so the model learns the identity mapping $f(x) \approx x$. For next digit generation (task 2), the target image is the image of the next digit, so the model learns the mapping $f(x) \approx x + 1$.
For task 3, we modified the architecture to accept two input images. Each image is passed through a shared encoder to obtain their respective latent representations, producing two latent vectors. Sharing weights ensures that both images are processed in the same manner and the learned features are consistent regardless of the input order. Upon obtaining the two latent vectors, we concatenate them to form a single combined latent representation of size $2 \times latent_dim$. This combined latent vector is then passed through a multilayer perceptron with ReLU activations to learn the relationship between the two input images and their sum. The output of the MLP is then split into two separate decoders, each responsible for generating one of the digits in the sum.

Task 1 (Reconstruction): $Encoder(Img) \rightarrow Latent \rightarrow Decoder(Latent) \rightarrow TargetImg$
Task 2 (Next Digit Generation): $Encoder(Img) \rightarrow Latent \rightarrow Decoder(Latent) \rightarrow TargetImg(nextdigit)$
Task 3 (Sum Digit Generation): $Encoder(Img1), Encoder(Img2) \rightarrow Latent1, Latent2$ $\rightarrow Concat(Latent1, Latent2) \rightarrow MLP \rightarrow Decoder1(Latent), Decoder2(Latent) \rightarrow TargetImg(sumdigits)$

TABLE I: Autoencoder architectures for the three tasks

For all tasks, we used the Mean Squared Error (MSE) loss function to measure the difference between the generated images and the target images. The models were trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 64 for different numbers of epochs depending on the task. Also, we trained a separate CNN classifier on the MNIST dataset to evaluate the quality of the generated images by measuring how well the classifier can recognize the generated digits correctly. Last but not least, we must mention that all experiments were conducted on the Google Colab platform, utilizing a Tesla T4 GPU for accelerated training.

IV Results and Evaluation

The performance of the autoencoder models was evaluated using both qualitative and quantitative metrics. Qualitatively, we visually inspected the generated images to assess their realism and resemblance to the target digits. Quantitatively, we computed the Mean Squared Error (MSE) loss between the generated images and the target images. Additionally, we evaluated the classification accuracy of the generated images using the pre-trained CNN classifier.

A. Task 1: Reconstruction

For the reconstruction task, we trained the autoencoder for 25 epochs with a latent dimension of 64. The generated images closely resembled the original input images, with an average MSE loss of 0.028 on the test set. The classification accuracy of the generated images was 98.2% (the pre-trained CNN classifier correctly recognized 98.2% of the generated digits), indicating that the autoencoder effectively learned to reconstruct the digit images. The training lasted 92.55 seconds. The accuracy and MSE plots are shown in Figures 1 and 2 respectively.

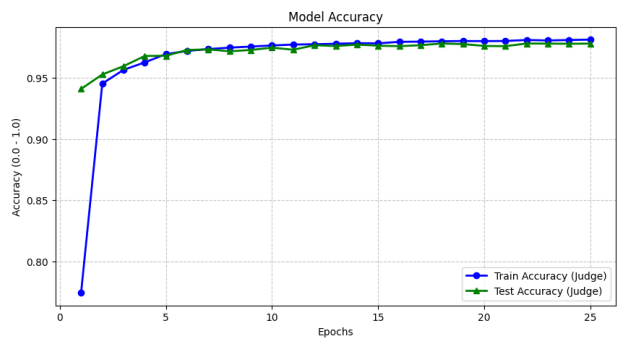

Fig. 1: Task 1: Training and Test Accuracy over Epochs

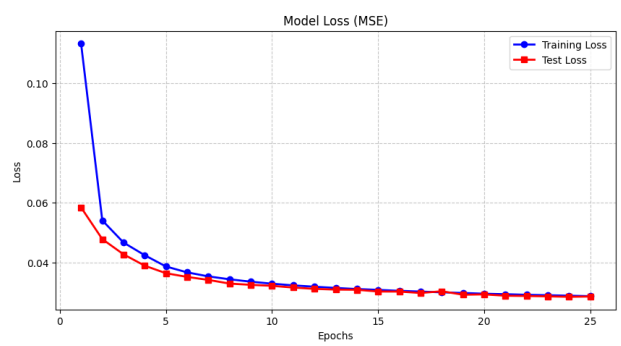

Fig. 2: Task 1: Training and Test MSE over Epochs

To further evaluate the performance of the autoencoder, we present its confusion matrix on the test set (Figure 3).

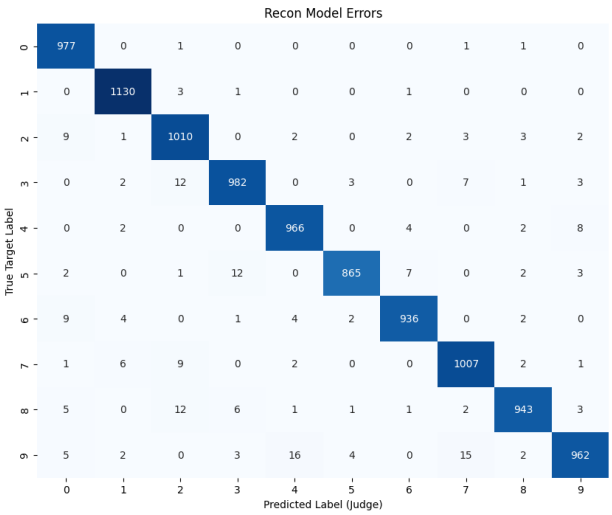

Fig. 3: Task 1: Confusion Matrix of Generated Images

The confusion matrix shows that the autoencoder was able to reconstruct most digits accurately, with only a few misclassifications occurring between similar-looking digits Specifically, digits like '5' or '8' and '9' had more misclassifications and the lowest accuracies, likely due to their more complex shapes.

Target Digit	Accuracy (%)
0	99.7
1	99.6
2	97.9
3	97.2
4	98.4
5	97.0
6	97.7
7	98.0
8	96.8
9	95.3

TABLE II: Task 1: Classification accuracy per target digit

We then compared the autoencoder reconstructions with PCA reconstructions retaining 90% of the variance. The autoencoder reconstructions were visually sharper and more detailed than the PCA reconstructions, which appeared more blurred. The average MSE loss for PCA reconstructions was 0.026, slightly lower than that of the autoencoder. The classification accuracy for PCA reconstructions was 98.49%, indicating that PCA also preserved the essential features of the digit images well. The major difference between the two methods is that PCA did not require any training time, while the autoencoder required 92.55 seconds of training.Below are some examples of original images, autoencoder reconstructions, and PCA reconstructions (Figure 4).

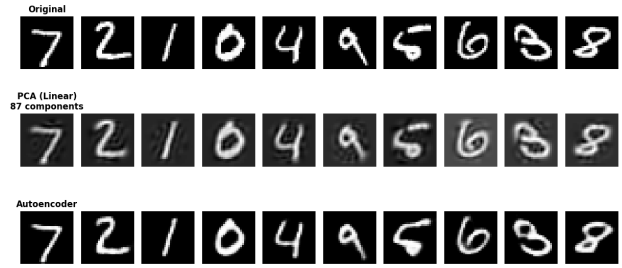

Fig. 4: Task 1: Original Images, Autoencoder Reconstructions, PCA Reconstructions

At last, we can conclude that both the autoencoder and PCA were effective in reconstructing the digit images, with the autoencoder providing slightly better visual quality at the cost of training time.

B. Task 2: Next Digit Generation

For the next digit generation task, we trained the autoencoder for 15 epochs with three different latent dimensions: 32, 64, and 128. The difference in performance across the different latent dimensions was minimal, with average MSE losses of 0.02166, 0.2164 and 0.2163 respectively. The classification accuracies were 98.19%, 98.39% and 98.58%, while the training times were pretty similar: 94.67s, 93.77s and 92.71s respectively. Here we present the results for latent dimension 128, which provided the best accuracy. The training and test accuracy and MSE plots are shown in Figures 5 and 6 respectively.

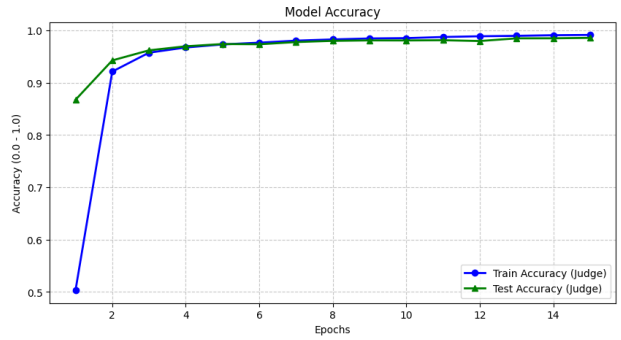

Fig. 5: Task 2: Training and Test Accuracy over Epochs

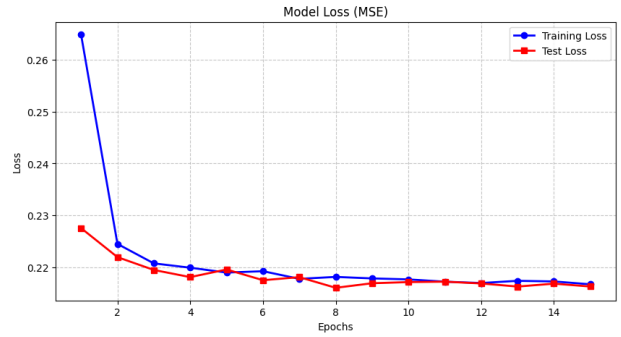

Fig. 6: Task 2: Training and Test MSE over Epochs

Some examples of input images and their corresponding generated next digit images are shown in the Figures below.

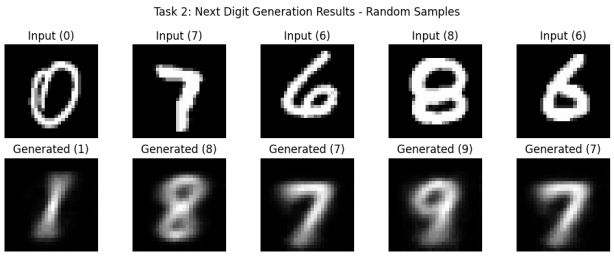

Fig. 7: Task 2: Input Images and Generated Next Digit Images

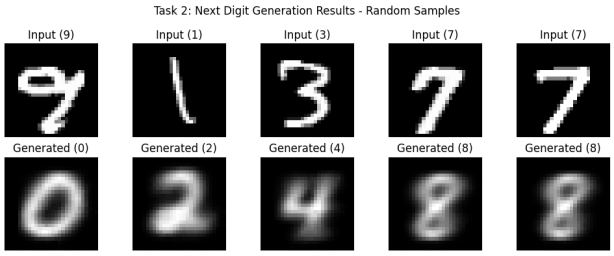

Fig. 8: Task 2: Input Images and Generated Next Digit Images

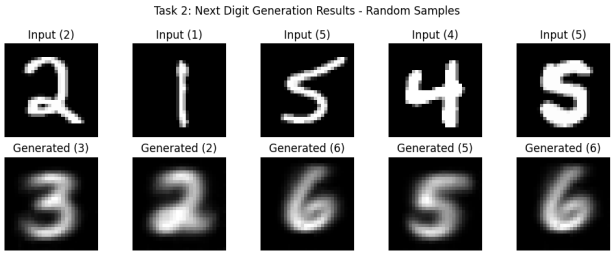

Fig. 9: Task 2: Input Images and Generated Next Digit Images

Although the generated images were generally blurry, they were still recognizable as the next digits. The high classification accuracy indicates that the autoencoder learned the mapping from each digit to its successor effectively.

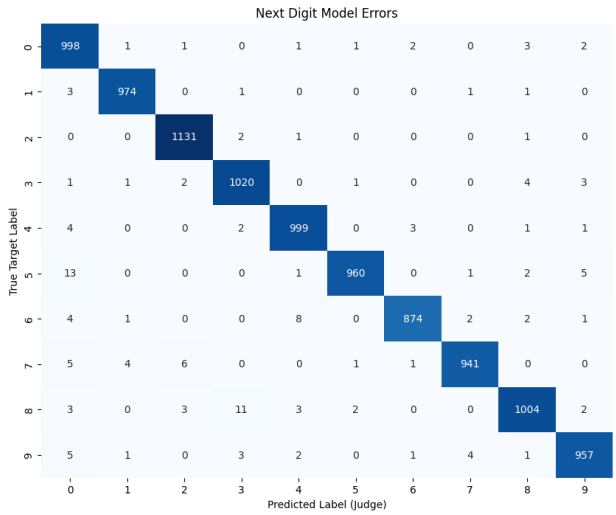

Fig. 10: Task 2: Confusion Matrix of Generated Images

Target Digit	Accuracy (%)
0	98.9
1	99.4
2	99.6
3	98.8
4	98.9
5	97.8
6	98.0
7	98.2
8	97.7
9	98.3

TABLE III: Task 2: Classification accuracy per target digit

The confusion matrix and accuracy per target digit (Figures 10 and Table above) show that the model performed well across all digits, with only a few misclassifications. Digits like '5' and '8' had slightly lower accuracies, but overall, the model demonstrated strong performance in generating the next digit images.

C. *Task 3: Sum Digit Generation*

For the sum digit generation task, we trained the modified autoencoder for 25 epochs with a latent dimension of 64 for each input image. The average MSE loss on the test set was around 0.41 and the classification accuracy of the generated images was 96.34% and the training lasted 333.62 seconds. The training and test accuracy and MSE plots are shown in Figures 11 and 12 respectively.

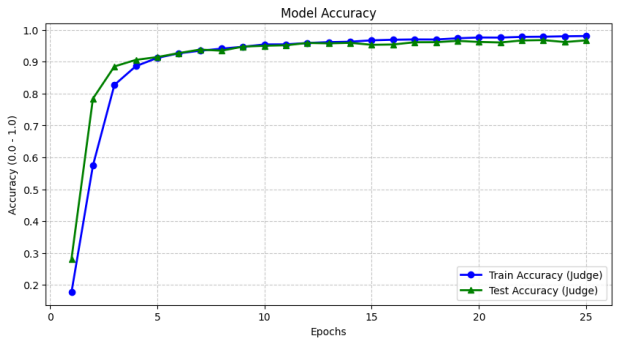

Fig. 11: Task 3: Training and Test Accuracy over Epochs

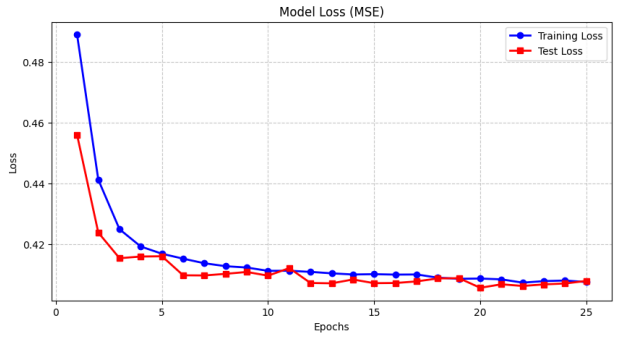

Fig. 12: Task 3: Training and Test MSE over Epochs

Based on the Figures above, we can see that the model was able to learn the relationship between the input digit images and their sum, as evidenced by the high classification accuracy of the generated images. Some examples of input image pairs and their corresponding generated sum digit images are shown in the Figures below.

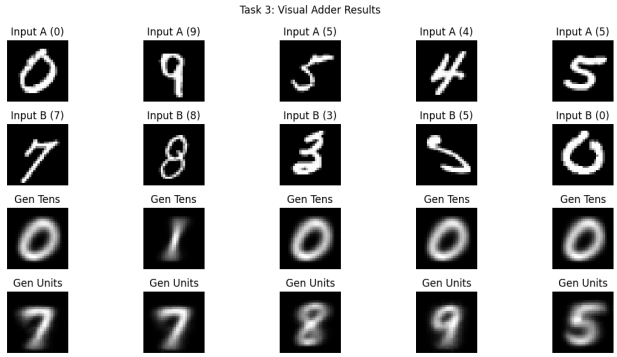

Fig. 13: Task 3: Input Image Pairs and Generated Sum Digit Images

In the examples above, the generated images accurately represent the digits of the sum, even when the sum results in a two-digit number. This demonstrates the model's ability to learn complex relationships between input images and generate appropriate outputs for both the tens and units place digits. Even though the generated images are a bit blurry, they are still easily recognizable as the correct digits. The confusion matrix and accuracy per target digit (Figures **??** and Table below) show that the model performed well across all digits, with only a few misclassifications.

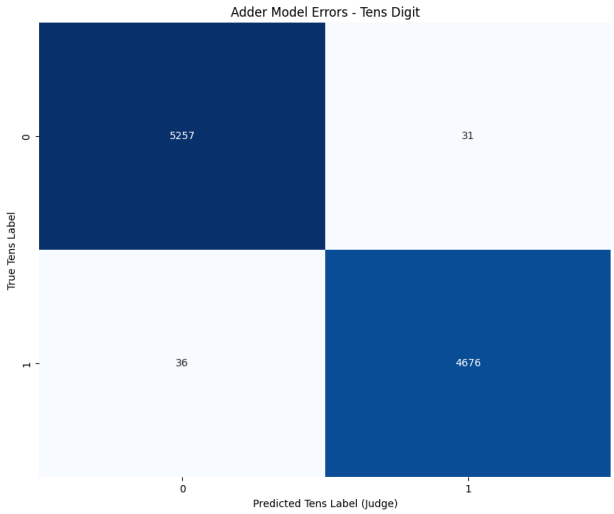

Fig. 14: Task 3: Confusion Matrix of Generated Images (Tens Place)

True Tens Digit	Accuracy (%)
0	99.4
1	99.2

TABLE IV: Classification accuracy per true tens digit

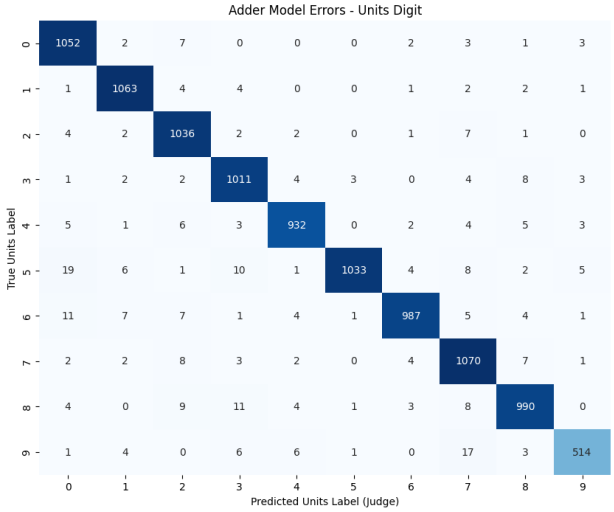

Fig. 15: Task 3: Confusion Matrix of Generated Images (Units Place)

True Units Digit	Accuracy (%)
0	98.3
1	98.6
2	98.2
3	97.4
4	97.0
5	94.9
6	96.0
7	97.4
8	96.1
9	93.1

TABLE V: Classification accuracy per true units digit

In the jupyter notebook, you can find examples of generated images for all possible addition combinations (0-9 + 0-9).

V Conclusion

In this report, we presented the implementation and evaluation of convolutional autoencoder neural networks for image generation using the MNIST dataset. We designed and trained autoencoder architectures for three tasks: reconstruction of input digit images, generation of the next digit image, and generation of sum digit images from pairs of input digit images. The models were trained using the Adam optimizer and evaluated using Mean Squared Error (MSE) loss and classification accuracy with a pre-trained CNN classifier. The results demonstrated that the autoencoders were effective in learning the mappings required for each task, achieving high classification accuracies of 98.2% for reconstruction, 98.58% for next digit generation, and 96.34% for sum digit generation. The generated images were generally recognizable as the intended digits, although some blurriness was observed, particularly in the next digit and sum digit generation tasks. Overall, this study highlights the potential of convolutional autoencoders for image generation tasks and provides a foundation for further exploration in this area.